Every model class supports the `find` method, which takes one or more primary key values. If given just one primary key, it returns an object containing data for the corresponding row (or throws an `ActiveRecord::RecordNotFound` exception). If given multiple primary key values, `find` returns an array of the corresponding objects. Note that in this case a `RecordNotFound` exception is raised if *any* of the IDs can't be found (so if the method returns without raising an error, the length of the resulting array will be equal to the number of IDs passed as parameters).

```
an_order = Order.find(27)   # find the order with id == 27

# Get a list of product ids from a form, then
# find the associated Products
product_list = Product.find(params[:product_ids])
```



David says:

To Raise or Not to Raise?

When you use a finder driven by primary keys, you're looking for a particular record. You expect it to exist. A call to `Person.find(5)` is based on our knowledge of the `people` table. We want the row with an ID of 5. If this call is unsuccessful—if the record with the ID of 5 has been destroyed—we're in an exceptional situation. This mandates the raising of an exception, so Rails raises `RecordNotFound`.

On the other hand, finders that use criteria to search are looking for a *match*. So, `Person.where(name: 'Dave').first` is the equivalent of telling the database (as a black box) "Give me the first person row that has the name Dave." This exhibits a distinctly different approach to retrieval; we're not certain up front that we'll get a result. It's entirely possible the result set may be empty. Thus, returning `nil` in the case of finders that search for one row and an empty array for finders that search for many rows is the natural, nonexceptional response.

Often, though, you need to read in rows based on criteria other than their primary key value. Active Record provides additional methods enabling you to express more complex queries.

SQL and Active Record

To illustrate how Active Record works with SQL, pass a string to the `where` method call corresponding to a SQL `where` clause. For example, to return a list of all orders for Dave with a payment type of "po," we could use this: